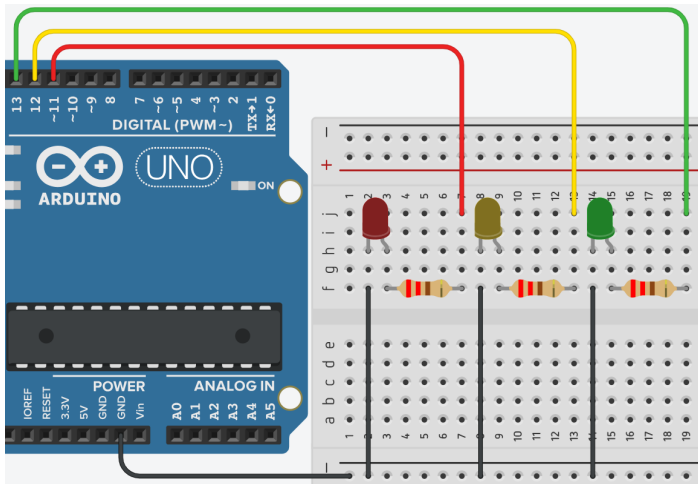


Semáforo - Declaración de Variables



Explicación del código

Este programa simula el comportamiento de un semáforo de tres luces (verde, amarillo, rojo) utilizando LEDs conectados a los pines 13 (verde), 12 (amarillo) y 11 (rojo). La secuencia es la estándar: **verde** → **amarillo** → **rojo** → (**repite**). Los tiempos son: verde 2 segundos, amarillo 1 segundo, rojo 2 segundos. Se emplean variables para los pines y los tiempos de espera, facilitando futuros ajustes.

1. Declaración de variables globales

```
//Bloque de Declaración
const int led_red = 11;
int led_yellow = 12;
int led_green = 13;
int espera1s = 1000;
int espera2s = 2000;
```

- `const int led_red = 11;` → constante para el LED rojo (pin 11). Al ser `const`, no puede modificarse accidentalmente.
- `int led_yellow = 12;` → variable para el LED amarillo (pin 12).
- `int led_green = 13;` → variable para el LED verde (pin 13).
- `int espera1s = 1000;` → 1000 ms = 1 segundo, usado para el amarillo.
- `int espera2s = 2000;` → 2000 ms = 2 segundos, usado para el verde y el rojo.
- **Nota:** Aunque `led_yellow` y `led_green` no se declaran como `const`, en la práctica no cambian. Se podrían declarar `const` también por claridad.

2. Configuración `setup()`

```
void setup()
{
  pinMode(led_red, OUTPUT);
  pinMode(led_yellow, OUTPUT);
  pinMode(led_green, OUTPUT);
}
```

- Configura los tres pines como salidas digitales, necesarias para controlar los LEDs.

3. Bucle principal `loop()`

```
void loop()
{
    //Semáforo
    digitalWrite(led_green, HIGH);
    delay(espera2s);
    digitalWrite(led_green, LOW);
    digitalWrite(led_yellow, HIGH);
    delay(espera1s);
    digitalWrite(led_yellow, LOW);
    digitalWrite(led_red, HIGH);
    delay(espera2s);
    digitalWrite(led_red, LOW);
    //
}
```

Secuencia paso a paso:

1. **Verde encendido** → `digitalWrite(led_green, HIGH)`
 - Duración: `delay(espera2s)` = 2000 ms (2 segundos).
2. **Apagar verde** → `digitalWrite(led_green, LOW)`
3. **Encender amarillo** → `digitalWrite(led_yellow, HIGH)`
 - Duración: `delay(espera1s)` = 1000 ms (1 segundo).
4. **Apagar amarillo** → `digitalWrite(led_yellow, LOW)`
5. **Encender rojo** → `digitalWrite(led_red, HIGH)`
 - Duración: `delay(espera2s)` = 2000 ms (2 segundos).
6. **Apagar rojo** → `digitalWrite(led_red, LOW)`
7. El `loop()` se repite, volviendo a encender el verde.

Ciclo completo: 2s (verde) + 1s (amarillo) + 2s (rojo) = **5 segundos**.

Comportamiento real de un semáforo:

- En la mayoría de semáforos, el amarillo aparece después del verde, y el rojo después del amarillo.
- El rojo suele durar más que el verde, pero aquí se ha simplificado con la misma duración (2s) para simetría.

Código completo para copiar y pegar

```
// Semáforo - Declaración de Variables
// Resistencias de 220 ohm en serie con cada LED

const int led_red = 11;
int led_yellow = 12;
int led_green = 13;
int espera1s = 1000;
int espera2s = 2000;
```

```

void setup()
{
  pinMode(led_red, OUTPUT);
  pinMode(led_yellow, OUTPUT);
  pinMode(led_green, OUTPUT);
}

void loop()
{
  // Secuencia de semáforo
  digitalWrite(led_green, HIGH);
  delay(espera2s);
  digitalWrite(led_green, LOW);

  digitalWrite(led_yellow, HIGH);
  delay(espera1s);
  digitalWrite(led_yellow, LOW);

  digitalWrite(led_red, HIGH);
  delay(espera2s);
  digitalWrite(led_red, LOW);
}

```

Enlace al simulador

[Código en Tinkercad](#)

Preguntas teóricas

1. ¿Por qué el semáforo enciende el verde, luego el amarillo, luego el rojo? ¿Qué pasaría si se invirtiera el orden (rojo → amarillo → verde)?
2. En este código, el rojo se enciende inmediatamente después de apagar el amarillo. En un semáforo real, ¿hay un pequeño tiempo con todas las luces apagadas? ¿Cómo se podría implementar ese “todo apagado” intermedio?
3. ¿Qué ventaja tiene usar `espera1s` y `espera2s` en lugar de escribir `1000` y `2000` directamente? Menciona al menos dos ventajas.
4. Si se cambia `espera2s` a 3000 y `espera1s` a 500, ¿cuánto dura ahora el ciclo completo del semáforo? ¿Cómo afecta eso a la percepción del semáforo?
5. ¿Es necesario declarar `led_yellow` y `led_green` como `const`? ¿Qué diferencia práctica hay entre declararlos `const` o no en este programa específico?

Ejercicios prácticos (modificar el código y anotar cambios)

Instrucciones: Para cada ejercicio, copia el código original, realiza la modificación indicada, carga el programa en el simulador (o en Arduino real) y describe cómo cambia el comportamiento del circuito.

Ejercicio 1

Intercambia los tiempos: que el verde dure 1 segundo, el amarillo 2 segundos y el rojo 1 segundo (usando las variables `espera1s` y `espera2s` de forma adecuada).

Pregunta: ¿El semáforo sigue siendo funcional? ¿Qué peligro podría tener un semáforo con amarillo tan largo?

Ejercicio 2

Añade un **LED azul** (pin 10) que funcione como "luz peatonal" que se encienda únicamente cuando el semáforo está en rojo, y se apague cuando el rojo se apaga.

Pregunta: ¿Cómo se comporta el azul? ¿Se enciende durante todo el tiempo que el rojo está encendido?

Ejercicio 3

Modifica el programa para que el semáforo tenga el siguiente patrón típico de algunos países:

- Verde 3 segundos
- Amarillo 1 segundo
- Rojo 3 segundos
- **Luego, antes de volver a verde, que el amarillo parpadee 3 veces (0.5 s encendido, 0.5 s apagado).**

Pregunta: ¿Cómo logras el parpadeo? ¿Qué duración total tiene ahora el ciclo?

Ejercicio 4

Crea dos semáforos sincronizados: uno en los pines 11,12,13 (el original) y otro en los pines 8,9,10. El segundo semáforo debe tener el **estado opuesto** al primero: cuando el primero está en verde, el segundo está en rojo; cuando el primero está en rojo, el segundo está en verde. El amarillo de ambos debe coincidir en el mismo instante (ambos amarillos juntos).

Pregunta: Describe la secuencia de luces para ambos semáforos. ¿Qué relación temporal hay entre ellos?

Ejercicio 5

Reemplaza todas las instrucciones de encendido/apagado por una máquina de estados usando una variable `estado` y `switch-case`. Debes tener estados: VERDE, AMARILLO, ROJO. Cada estado debe durar su tiempo correspondiente y luego pasar al siguiente.

Pregunta: ¿Qué ventaja tiene esta estructura frente a la secuencia lineal con `delay()`? ¿Cómo afecta a la capacidad de leer sensores mientras se espera?

Entregar las respuestas a las preguntas teóricas y la descripción de los cambios observados en cada ejercicio.